

AUTOMATIC WASHING MACHINE

A project report submitted in partial fulfilment of the requirements for the award of the Degree

of

BACHELOR OF TECHNOLOGY

In

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted By

REDDY SOMA SAI SATYA SNIGDHA (BU21EECE0100200)

MONISHA H S (BU22EECE0100439)

DHEERAJ LAKSHMAN (BU21EECE0100305)

ALLI GOPI (BU21EECE0100161)

DARSHAN P (2023007316)

Under the esteemed guidance of

Dr Arun Kumar Manoharan

**Associate Professor, Department of EECE, GST,
GITAM DEEMED TO BE UNIVERSITY, BENGALURU**

Dr Karthick S

**Associate Professor, Department of EECE, GST,
GITAM DEEMED TO BE UNIVERSITY, BENGALURU**



**DEPARTMENT OF ELECTRICAL, ELECTRONICS AND
COMMUNICATION ENGINEERING**

**GITAM INSTITUTE OF
TECHNOLOGY GITAM**

(Deemed to be University)

**(Estd. u/s 3 of UGC act 1956 & Accredited by NAAC with "A+" Grade
BENGALURU – 561203 (2024))**

CONTENTS

1.Introduction

2.Overview of Automatic Washing Machine

3.Methodology

- ❖ **Software used**
- ❖ **Hardware used**
- ❖ **Block Diagram**

4.Result

- ❖ **RTL (Register Transfer Level)**
- ❖ **Simulation Results**
- ❖ **FPGA Implementation**

5.Conclusion

6.Reference and Link

1. Introduction to Automatic Washing Machine

Automatic washing machines have revolutionized the way we approach household chores, offering a blend of convenience, efficiency, and advanced technology. These appliances are designed to handle the entire washing process, from soaking and washing to rinsing and spinning, without the need for constant human intervention. By automating these tasks, automatic washing machines save time, reduce physical effort, and ensure consistent washing performance.

Modern automatic washing machines incorporate a variety of advanced features, including programmable settings, multiple wash cycles, and intelligent sensors. These features enable the machines to adapt to different fabric types, load sizes, and soil levels, providing optimal care for a wide range of garments. The integration of smart technology allows for remote control and monitoring, adding another layer of convenience for users.

This project delves into the key components and functionalities of automatic washing machines, exploring their design, operation, and technological advancements. It aims to provide a comprehensive understanding of how these machines work, the benefits they offer, and the future trends in washing machine technology. By examining the automatic washing machine's development and innovations, we gain insights into the evolution of home appliances and their impact on daily life.

2. Overview of Automatic Washing Machine

Automatic washing machines have become an indispensable part of modern households, offering unparalleled convenience and efficiency in laundry care. Unlike traditional manual washing methods, these machines are designed to perform all washing tasks automatically, including filling the tub with water, agitating the clothes, rinsing, and spinning them dry. This automation not only saves time and effort but also ensures a consistent and thorough cleaning process.

3. Methodology of Automatic Washing Machine

This project involves the design, simulation, and analysis of an automatic washing machine system. The methodology includes the use of both software and hardware components to simulate the washing machine's operations and evaluate its performance.

❖ Software used:

Xilinx Vivado Software: Vivado, developed by Xilinx, is an advanced software suite tailored for FPGA and SoC design. It supports design entry via schematics, Verilog, VHDL, and high-level synthesis, optimizing designs through efficient place-and-route. Vivado enables bitstream programming, provides robust debugging tools, supports IP integration, and facilitates hardware validation through simulation. This software is crucial for designing and simulating the control logic for the washing machine.

❖ Hardware used:

Digilent Nexys 4 DDR Artix-7 FPGA Board: The Nexys 4 board is a complete, ready-to-use digital circuit development platform based on the Artix-7™ Field Programmable Gate Array (FPGA) from Xilinx. With its large, high-capacity FPGA (Xilinx part number XC7A100T-1CSG324C), generous external memories, and collection of USB, Ethernet, and other ports, the Nexys 4 can host designs ranging from introductory combinational circuits to powerful embedded processors. Several built-in peripherals, including an accelerometer, temperature sensor, MEMS digital microphone, a speaker amplifier, and a lot of I/O devices allow the Nexys 4 to be used for a wide range of designs without needing any other components.

Digilent Basys 3™ Artix-7 FPGA Trainer Board:

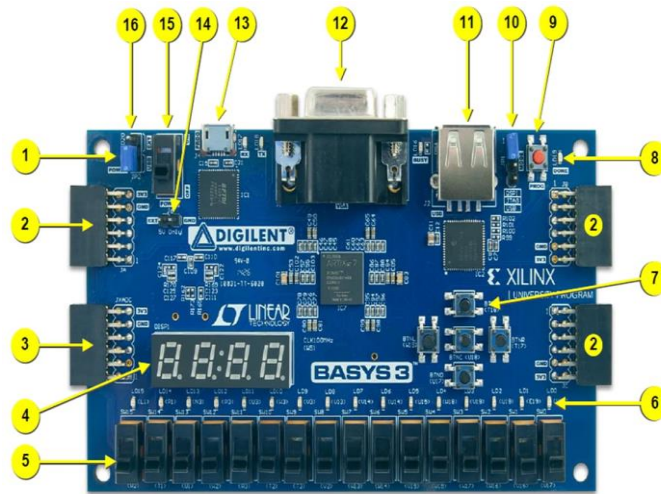
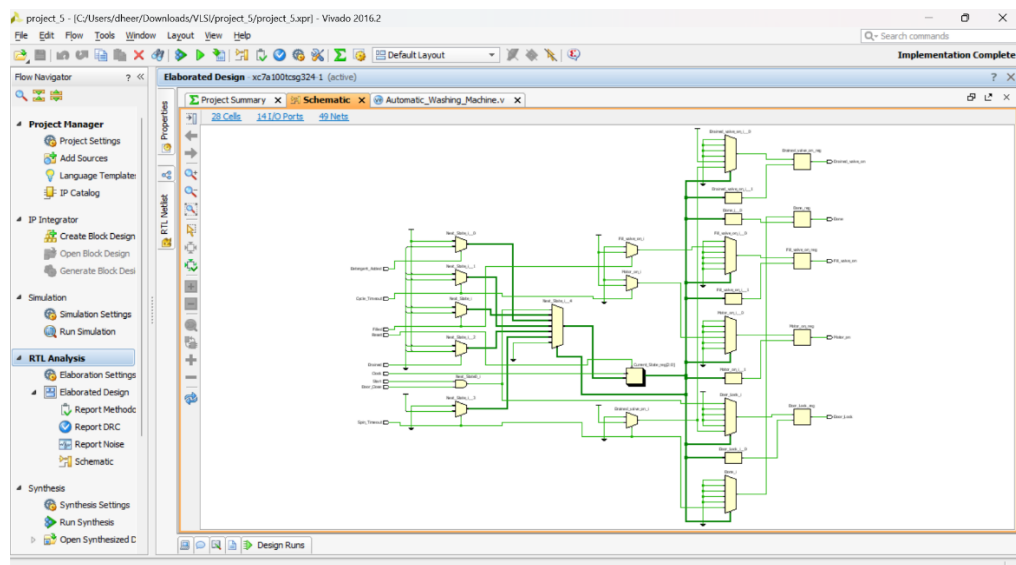


Figure 1. Basys 3 FPGA board with callouts.

Callout	Component Description	Callout	Component Description
1	Power good LED	9	FPGA configuration reset button
2	Pmod port(s)	10	Programming mode jumper
3	Analog signal Pmod port (XADC)	11	USB host connector
4	Four digit 7-segment display	12	VGA connector
5	Slide switches (16)	13	Shared UART/ JTAG USB port
6	LEDs (16)	14	External power connector
7	Pushbuttons (5)	15	Power Switch
8	FPGA programming done LED	16	Power Select Jumper

❖ Block Diagram:



❖ Enter the Code:

```
`timescale 1ns / 10ps

module Automatic_washing_Machine(
    input Clock, Reset, Start, Door_Close, Filled, Drained, Detergent_Added,
    Cycle_Timeout, Spin_Timeout,
    output reg Motor_on, Fill_valve_on, Drained_valve_on, Door_Lock, Done
);

    // Define states
    parameter Check_Door=3'b000;
    parameter Fill_Water=3'b001;
    parameter Add_Detergent=3'b010;
    parameter Cycle=3'b011;
    parameter Drain_Water=3'b100;
    parameter Spin=3'b101;

    // Define state registers
    reg [2:0] Current_State, Next_State;

    // State transition and output logic
    always @(posedge Clock or negedge Reset) begin
        if (Reset == 0)
            Current_State <= Check_Door;
        else
            Current_State <= Next_State;
    end

    always @* begin
        case(Current_State)
            Check_Door:
                if(Start==1 && Door_Close==1) begin
                    Next_State = Fill_Water;
                    Motor_on = 0;
                    Fill_valve_on = 0;
                    Drained_valve_on = 0;
                    Door_Lock = 1;
                    Done = 0;
                end
            else begin
                Next_State = Check_Door;
                Motor_on = 0;
                Fill_valve_on = 0;
                Drained_valve_on = 0;
                Door_Lock = 0;
                Done = 0;
            end
        end
        Fill_Water:
            if(Filled==1) begin
```

```

        Next_State = Add_Detergent;
        Motor_on = 0;
        Fill_valve_on = 0;
        Drained_valve_on = 0;
        Door_Lock = 1;
        Done = 0;
    end
else begin
    Next_State = Fill_Water;
    Motor_on = 0;
    Fill_valve_on = 1;
    Drained_valve_on = 0;
    Door_Lock = 1;
    Done = 0;
end
Add_Detergent:
    if(Detergent_Added==1) begin
        Next_State = Cycle;
        Motor_on = 0;
        Fill_valve_on = 0;
        Drained_valve_on = 0;
        Door_Lock = 1;
        Done = 0;
    end
else begin
    Next_State = Add_Detergent;
    Motor_on = 0;
    Fill_valve_on = 0;
    Drained_valve_on = 0;
    Door_Lock = 1;
    Done = 0;
end
Cycle:
    if(Cycle_Timeout==1) begin
        Next_State = Drain_Water;
        Motor_on = 0;
        Fill_valve_on = 0;
        Drained_valve_on = 0;
        Door_Lock = 1;
        Done = 0;
    end
else begin
    Next_State = Cycle;
    Motor_on = 1;
    Fill_valve_on = 0;
    Drained_valve_on = 0;
    Door_Lock = 1;
    Done = 0;
end
Drain_Water:
    if(Drained==1) begin

```

```

        Next_State = Spin;
        Motor_on = 0;
        Fill_valve_on = 0;
        Drained_valve_on = 1;
        Door_Lock = 1;
        Done = 0;
    end
    else begin
        Next_State = Drain_Water;
        Motor_on = 0;
        Fill_valve_on = 0;
        Drained_valve_on = 1;
        Door_Lock = 1;
        Done = 0;
    end
    Spin:
    if(Spin_Timeout==1) begin
        Next_State = Check_Door;
        Motor_on = 0;
        Fill_valve_on = 0;
        Drained_valve_on = 0;
        Door_Lock = 0;
        Done = 1;
    end
    else begin
        Next_State = Spin;
        Motor_on = 0;
        Fill_valve_on = 0;
        Drained_valve_on = 1;
        Door_Lock = 1;
        Done = 0;
    end
    default:
        Next_State = Check_Door;
    endcase
end
endmodule

```

❖ Test Bench Code:

```

// washing_machine.v
module washing_machine (
    input wire clk,
    input wire reset,
    input wire start,
    input wire [1:0] mode, // 2-bit mode selector for different wash cycles
    input wire water_level_sensor,
    input wire temperature_sensor,

```



```

    input wire load_sensor,
    input wire vibration_sensor,
    output reg water_inlet_valve,
    output reg motor,
    output reg detergent_dispenser,
    output reg drain_pump
);

// Define states
typedef enum reg [3:0] {
    IDLE,
    FILL,
    WASH,
    RINSE,
    SPIN,
    DRAIN
} state_t;

state_t current_state, next_state;

// State transition logic
always @(posedge clk or posedge reset) begin
    if (reset)
        current_state <= IDLE;
    else
        current_state <= next_state;
end

// Next state logic
always @(*) begin
    case (current_state)
        IDLE: if (start) next_state = FILL; else next_state = IDLE;
        FILL: if (water_level_sensor) next_state = WASH; else next_state = FILL;
        WASH: next_state = RINSE; // Simplified transition
        RINSE: next_state = SPIN; // Simplified transition
        SPIN: if (vibration_sensor) next_state = DRAIN; else next_state = SPIN;
        DRAIN: next_state = IDLE;
        default: next_state = IDLE;
    endcase
end

// Output logic
always @(posedge clk or posedge reset) begin
    if (reset) begin
        water_inlet_valve <= 0;
        motor <= 0;
        detergent_dispenser <= 0;
        drain_pump <= 0;
    end else begin
        case (current_state)
            FILL: water_inlet_valve <= 1;

```

```

WASH: begin
    water_inlet_valve <= 0;
    motor <= 1;
    detergent_dispenser <= 1;
end
RINSE: motor <= 1;
SPIN: motor <= 1;
DRAIN: drain_pump <= 1;
default: begin
    water_inlet_valve <= 0;
    motor <= 0;
    detergent_dispenser <= 0;
    drain_pump <= 0;
end
endcase
end
end

endmodule

```

4.Result of Automatic Washing Machine

❖ RTL (Register Transfer Level):

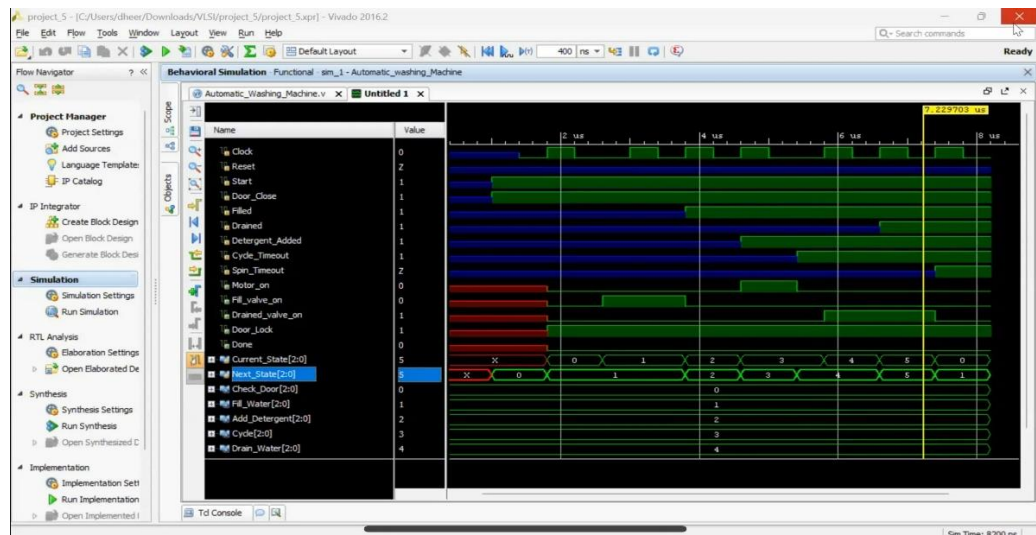
The RTL design is a critical step in the development of the automatic washing machine's control system. RTL involves the creation of a detailed hardware description of the system using a Hardware Description Language (HDL) such as Verilog or VHDL. This design stage translates the high-level algorithm of the washing machine into a low-level representation that can be synthesized into an FPGA.

In this project, the RTL design includes modules for water level control, motor control, cycle timing, and sensor interfacing. Each module is described in Verilog, ensuring that the entire washing process is efficiently managed and synchronized. The RTL design is verified through simulation to ensure it meets all functional requirements before moving on to the physical implementation on the FPGA.

❖ Simulation Results:

Simulation plays a crucial role in validating the design of the automatic washing machine system. Using Xilinx Vivado, we performed extensive simulations to verify the correctness of the RTL design. The simulation environment mimics real-world scenarios, allowing us to test various washing machine operations such as filling, washing, rinsing, and spinning under different conditions.

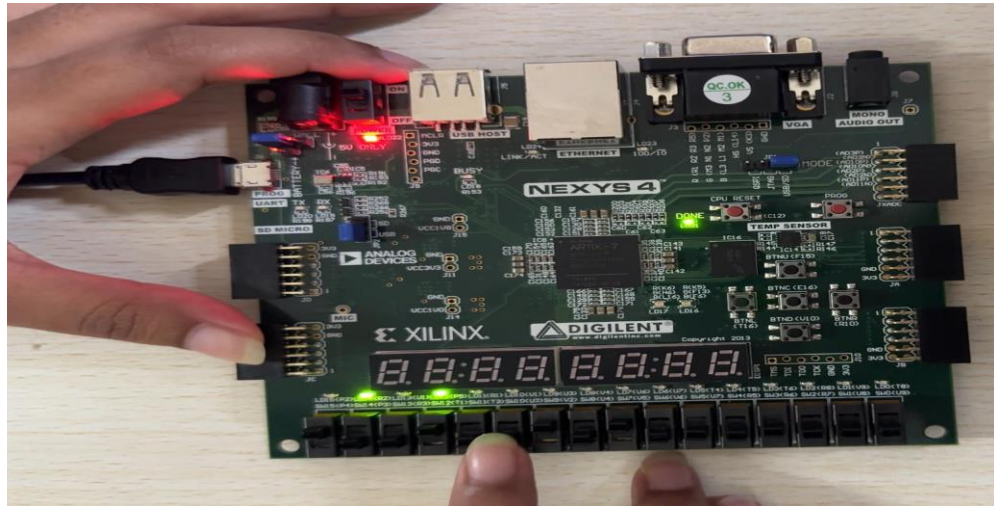
The simulation results confirmed that the control logic operates correctly, responding appropriately to sensor inputs and managing the washing cycles as intended. Key performance metrics such as cycle time, water usage, and energy.



❖ FPGA Implementation:

The final step involves implementing the verified RTL design on the Digilent Nexys 4 DDR Artix-7 FPGA board. This process includes synthesizing the RTL code into a bitstream that configures the FPGA to perform the specified control tasks. The FPGA implementation provides a platform to test the washing machine control system in a real hardware environment.

The Nexys 4 board's rich set of I/O peripherals and built-in features facilitate comprehensive testing of the washing machine system. The FPGA's flexibility allows for adjustments and optimizations based on hardware test results, ensuring the final design is robust and reliable.



5.Conclusion

The design, simulation, and implementation of an automatic washing machine control system demonstrate the feasibility and effectiveness of using FPGA technology for complex household appliances. The project highlights the importance of RTL design and simulation in developing reliable and efficient control systems. Future work may involve optimizing the design for power consumption and exploring additional features such as remote control and machine learning-based cycle optimization.

6.Reference and Link

❖ Drive link for the video tutorial of this project:

<https://drive.google.com/drive/folders/1qVRmLLs0TxHpfofTdiQgyJT6-WxqxILW>

❖ Xilinx Vivado link to start:

https://digilent.com/reference/vivado/getting_started/start